

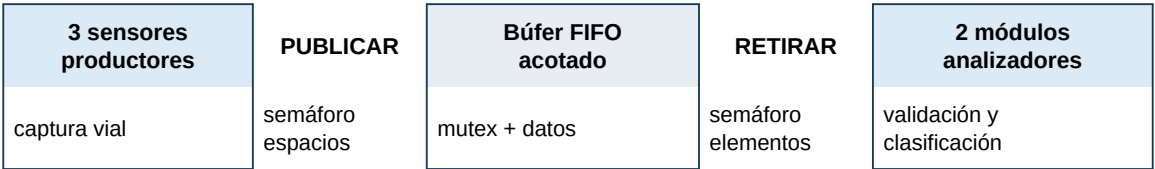
Relatoría técnica de concurrencia en el SIGET

Simulación productor-consumidor con semáforos en Python

Resumen y conclusión principal

La solución simula el flujo de información de una infraestructura vial: tres sensores producen lecturas de velocidad, ocupación y volumen; dos módulos las validan y clasifican; y un supervisor observa el avance. Se ejecutan seis hilos concurrentes sobre un búfer compartido. Dos semáforos controlan la cantidad de espacios y elementos, mientras mutex independientes protegen el búfer, las métricas y la consola. La verificación final confirma que cada identificador producido fue procesado exactamente una vez. Por ello, el diseño mantiene integridad y progreso sin espera activa ni interbloqueo en el escenario simulado.

Arquitectura de la simulación



Cada productor usa una semilla independiente y publica objetos inmutables. Los consumidores forman un pool: cada lectura es retirada por uno de ellos y se clasifica como NORMAL, CONGESTION o EXCESO_VELOCIDAD. El supervisor consulta periódicamente cantidades producidas, procesadas y almacenadas, una tarea distinta que evidencia responsividad mientras continúa el trabajo principal.

Mecanismos de concurrencia

Mecanismo	Uso en la solución	Propiedad obtenida
Semáforo espacios	Inicia en la capacidad del búfer; cada productor adquiere antes de insertar.	Impide desbordamiento y aplica espera bloqueante.
Semáforo elementos	Inicia en cero; cada consumidor adquiere antes de retirar.	Impide leer un búfer vacío.
Mutex	Protege append, popleft, métricas y salida de consola.	Evita carreras, corrupción e impresiones mezcladas.
Eventos y FIN	Un evento libera el inicio; un marcador FIN por consumidor cierra el sistema.	Arranque coordinado y terminación sin bloqueo permanente.

Secuencia de operación

Al publicar, el productor adquiere un permiso de espacios, entra al mutex, inserta en el deque FIFO, libera el mutex y anuncia un elemento. El consumidor realiza la secuencia complementaria. Nunca se duerme ni se analiza dentro de la sección crítica; así disminuye la contención y otros hilos avanzan. Cuando terminan los productores, el hilo principal inserta dos marcadores FIN. Cada consumidor retira uno después de procesar las lecturas previas y finaliza ordenadamente; luego se detiene el supervisor y se validan los invariantes.

Relación con el sistema operativo

La concurrencia no implica que todos los hilos ejecuten instrucciones exactamente al mismo instante: el planificador del sistema operativo reparte CPU y puede intercalarlos o ejecutarlos en paralelo si existen varios núcleos. Los semáforos expresan dependencias de disponibilidad; cuando un permiso no existe, el hilo queda bloqueado y el sistema operativo puede asignar la CPU a trabajo útil. El mutex limita el acceso exclusivo a la estructura compartida. Esta combinación favorece utilización eficiente, evita espera activa y mantiene pequeñas las regiones serializadas, aunque el intérprete de Python pueda restringir el paralelismo de tareas intensivas de CPU.

Análisis de fiabilidad y gestión de recursos

Riesgos de concurrencia considerados y respuesta del diseño

Riesgo	Cómo podría ocurrir	Mitigación implementada
Condición de carrera	Dos hilos modifican simultáneamente el deque o las métricas.	Mutex exclusivo y datos LecturaTrafico inmutables.
Interbloqueo	Un hilo conserva un recurso mientras espera otro, o un consumidor queda esperando al final.	Orden fijo adquirir-operar-liberar, secciones críticas breves y un FIN por consumidor.
Inanición	Un actor no consigue acceder al búfer durante un periodo indefinido.	Operaciones finitas, liberación inmediata de permisos y ningún mutex durante las pausas simuladas.
Pérdida o duplicación	Una lectura se sobrescribe, se procesa dos veces o desaparece.	FIFO acotado e igualdad final de los contadores de identificadores únicos.

Evidencia obtenida

En la ejecución de demostración --lecturas 12 --capacidad 3, los nombres de los seis actores aparecieron intercalados y el supervisor informó el avance sin detener productores ni consumidores. Se produjeron y procesaron 36 lecturas: 19 normales, 14 de congestión y 3 de exceso de velocidad. El búfer pequeño provocó 26 esperas de productores y 2 de consumidores, lo cual demuestra que los semáforos bloquearon y reanudaron actores según la disponibilidad, sin consumir CPU mediante ciclos de sondeo.

VERIFICACIÓN DE INTEGRIDAD: APROBADA | 36 producidas = 36 procesadas

Además, cuatro pruebas automáticas validaron reglas de clasificación, rechazo de capacidad inválida y conservación de nueve lecturas bajo mayor presión (capacidad dos). La evidencia en video debe mostrar la salida intercalada, la ocupación del búfer, las esperas, el cierre de ambos analizadores, la verificación aprobada y el resultado de python -m unittest -v.

Reflexión, ventajas y límites

El patrón productor-consumidor desacopla la velocidad de captura y análisis: una ráfaga corta se absorbe en el búfer y la presión de retorno frena sensores simulados cuando se alcanza el límite. Esto mejora estabilidad y uso de memoria frente a una cola ilimitada. También introduce decisiones: un búfer muy pequeño aumenta bloqueos; uno muy grande incrementa memoria y latencia. La solución evita deadlock por su orden simple de adquisición, pero la equidad exacta depende de la planificación de hilos; en cargas permanentes sería necesario medir latencias y aplicar prioridades o particiones.

En un SIGET real, los hilos y la memoria compartida no bastan para fallos de red, reinicios o datos fuera de orden. La evolución natural incluiría colas persistentes, marcas de tiempo confiables, reintentos idempotentes, límites de latencia, métricas por intersección y redundancia. Aun con esa limitación, la práctica demuestra el principio esencial: la fiabilidad no surge de ejecutar más tareas, sino de definir quién puede acceder a cada recurso, bajo qué condición y cómo se verifica el resultado.

Conclusión

La implementación cumple el escenario SIGET, supera el mínimo de tres tareas concurrentes, usa semáforos y exclusión mutua de forma explícita, previene carreras e interbloqueos y produce evidencia verificable. La separación entre captura, análisis y supervisión refleja responsabilidades reales y muestra cómo la sincronización correcta protege una infraestructura urbana crítica.